

Lab-one: MapReduce

论文

☒ Mapreduce

☒ Gfs

☒ Paxos

MapReduce

介绍

MapReduce将所有计算抽象为**Map（映射）**和**Reduce（归约）**两个用户自定义函数，以**键值对（Key/Value）**为核心数据格式，完成从输入到输出的转换，核心流程为：**输入K1,V1 → Map → 中间K2,V2 → 分组聚合 → Reduce → 输出K2,V2**。

- **Map函数**：接收输入键值对 $\langle k1, v1 \rangle$ ，处理后生成一组中间键值对 $\langle k2, v2 \rangle$ ，实现数据的初步处理与转换；
- **Reduce函数**：接收中间键 $k2$ 和其对应的所有值列表 $list(v2)$ ，对值进行聚合（求和、拼接、排序等），生成最终的键值对 $\langle k2, v2 \rangle$ ，实现数据的归并处理。

应用场景

- 分布式Grep：Map匹配符合正则的行，Reduce直接转发结果；
- URL访问频次统计：Map输出 $\langle URL, 1 \rangle$ ，Reduce求和；
- 反向网页链接图：Map输出 $\langle \text{目标URL}, \text{源URL} \rangle$ ，Reduce拼接源URL列表；
- 倒排索引：Map输出 $\langle \text{单词}, \text{文档ID} \rangle$ ，Reduce排序并拼接文档ID列表；
- 分布式排序：Map提取排序键，Reduce直接转发，依托框架的分区与排序能力完成全局排序。

集群环境

论文针对Google的**商用机集群环境**（Linux、千兆网卡、本地IDE磁盘、GFS分布式文件系统）设计了MapReduce的实现方案，核心由**Master（主节点）**和**Worker（工作节点）**构成，分为Map和Reduce两个阶段，共7个核心执行步骤：

1. **数据分片**：将输入文件切分为M个分片（16-64MB/片），集群启动多个程序副本；

2. **任务分配**：一个副本作为Master，其余为Worker，Master为Worker分配M个Map任务和R个Reduce任务；
3. **Map任务执行**：Worker读取对应分片，解析键值对并调用Map函数，中间结果缓存在内存；
4. **中间结果落盘**：内存中的中间结果按分区函数（如 $\text{hash}(\text{key}) \bmod R$ ）分为R个区域，定期写入本地磁盘，将文件位置上报Master；
5. **Reduce任务拉取数据**：Master将中间文件位置通知Reduce Worker，Worker通过RPC拉取所有对应分区的中间数据，拉取完成后按key排序；
6. **Reduce任务执行**：遍历排序后的中间数据，对每个唯一key调用Reduce函数，结果写入最终输出文件（每个Reduce任务对应一个输出文件）；
7. **任务完成**：所有Map和Reduce任务执行完毕后，Master唤醒用户程序，MapReduce调用结束。

（1）Master核心数据结构

Master维护所有任务的状态（**空闲/执行中/完成**）和执行节点信息，同时存储已完成Map任务的中间文件位置，作为Map和Reduce任务的信息桥梁，是整个框架的“大脑”。

（2）容错机制

集群中机器故障是常态，框架通过**重执行**实现容错，分为Worker和Master容错：

- **Worker容错**：Master定期Ping Worker，无响应则标记为故障；其已完成的Map任务需重执行（中间结果存在本地磁盘），执行中的Map/Reduce任务重置为空闲；重执行后，Master通知所有Reduce Worker从新节点拉取数据；
- **Master容错**：暂未实现主备切换，Master故障则整个计算终止，用户可重试；可通过定期检查点持久化Master状态，为后续优化预留空间；
- **语义保证**：若Map/Reduce函数是**确定性的**，分布式执行结果与单机串行执行结果完全一致，依托**原子提交**实现（任务输出先写入临时文件，完成后原子重命名为最终文件）。

（3）本地性优化

网络带宽是集群稀缺资源，框架依托GFS的**数据副本策略**（每个文件块3个副本）实现本地性：

- Master优先将Map任务调度到**存储了对应数据分片副本**的Worker节点；
- 若无，则调度到同一网络交换机的节点，最大化本地数据读取，减少跨节点网络传输。

（4）任务粒度与负载均衡

- Map任务数M远大于集群机器数，每个Worker执行多个Map任务，实现动态负载均衡；
- Reduce任务数R由用户指定，通常为机器数的小倍数；

- 实际部署中常采用M=200000、R=5000、2000台Worker的配置，兼顾调度效率与负载均衡。

(5) 备份任务（解决拖尾问题）

拖尾节点（Straggler）：少数机器因磁盘故障、CPU竞争、缓存禁用等原因，执行速度极慢，导致整个计算被拖慢。

解决方案：计算接近完成时，Master为剩余的执行中任务启动**备份任务**，主任务和备份任务任一完成则标记为任务完成，仅增加少量计算资源（约数%），但能显著缩短计算时间（如排序任务无备份时耗时增加44%）。

扩展

为提升框架的实用性和性能，论文在基础Map/Reduce之上增加了9项核心扩展，成为后续框架的通用设计：

1. **自定义分区函数**：默认用 $\text{hash}(\text{key}) \bmod R$ 分区，支持用户自定义（如按URL的主机名分区，让同一主机的URL落在同一Reduce任务）；
2. **键排序保证**：同一Reduce分区内的中间键值对按key升序排列，方便生成有序输出；
3. **Combiner函数**：对Map节点的中间结果做**局部聚合**（如单词计数中，Map节点将 $\langle \text{the}, 1 \rangle$ 聚合为 $\langle \text{the}, 100 \rangle$ ），减少网络传输数据量，通常复用Reduce函数代码；
4. **多输入输出格式**：支持文本、有序键值对等多种输入格式，用户可自定义Reader/Writer，也支持从数据库、内存结构读取数据；
5. **副作用处理**：允许生成辅助文件，要求用户保证操作的**原子性和幂等性**（如先写临时文件，完成后原子重命名）；
6. **跳过坏记录**：自动检测导致Map/Reduce崩溃的坏记录并跳过，避免因少量坏数据导致整个计算失败；
7. **本地执行模式**：支持在单机上串行执行，方便调试、性能分析和小规模测试；
8. **状态监控**：Master启动内置HTTP服务器，展示任务完成率、数据处理速率、节点故障信息等，支持用户监控计算进度；
9. **计数器**：用户可自定义计数器（如统计大写单词数），框架自动聚合所有节点的计数器值，也内置默认计数器（如输入/输出键值对数量），用于校验计算正确性。



一、核心设计理念与架构

1. 设计前提与突破

GFS 的设计源于对谷歌应用 workload 和技术环境的四大关键观察，彻底颠覆了传统文件系统的固有假设：

- **组件故障常态化**：由廉价硬件构成的集群中，磁盘、内存、网络等组件故障是常态，因此容错性和自动恢复必须内置为核心能力，而非附加功能。
- **文件规模巨型化**：常见文件多为 GB 级，TB 级数据集普遍存在，需优化大文件的存储与访问效率，而非传统的 KB 级小文件。
- **访问模式特定化**：文件操作以“追加写入”和“顺序读取”为主，随机写入极少，因此无需优化随机写性能，转而聚焦追加操作的原子性和吞吐量。
- **应用与接口协同设计**：通过松弛一致性模型和定制化接口（如原子追加），在简化文件系统实现的同时，降低应用层同步开销。

2. 三层架构设计

GFS 集群由**单一主节点（Master）**、**多个数据块服务器（Chunkserver）**和**多个客户端（Client）**构成，架构简洁且职责明确：

- **Master**：核心元数据管理者，存储三类关键信息——文件与数据块命名空间、文件到数据块的映射、数据块副本的物理位置。所有元数据常驻内存，保证操作高效，同时通过操作日志（Operation Log）和 checkpoint 实现持久化。
- **Chunkserver**：实际数据存储节点，将数据块以 Linux 本地文件形式存储。数据块默认大小为 64MB（远大于传统文件系统块大小），每个数据块会在不同机架的 Chunkserver 上保留 3 个副本（可配置），确保容错性。
- **Client**：提供文件系统 API 实现，与 Master 交互获取元数据（如数据块位置），但所有数据读写直接与 Chunkserver 通信，避免 Master 成为性能瓶颈。Client 不缓存数据块，仅缓存元数据以减少 Master 交互。

二、关键技术机制

1. 数据块管理：64MB 大尺寸设计

64MB 的数据块是 GFS 最具标志性的设计之一，其优势与权衡如下：

- **核心优势**：
 - a. 减少 Client 与 Master 的交互频率，单次元数据获取可支持大量连续读写；
 - b. 便于 Client 与 Chunkserver 建立持久 TCP 连接，降低网络开销；
 - c. 大幅缩减 Master 存储的元数据量（每个数据块仅需不到 64 字节元数据），支持内存全量存储。
- **潜在问题与解决方案**：小文件可能仅占用一个数据块，导致存储该数据块的 Chunkserver 成为热点。通过提高小文件副本数、错开批量任务启动时间等方式缓解。

2. 一致性模型：松弛但可控

GFS 采用松弛一致性模型，平衡系统复杂度与应用需求，核心规则如下：

- **命名空间操作原子性**：文件创建、删除等操作由 Master 独占处理，通过命名空间锁和操作日志保证全局有序。
- **数据突变后的状态**：
 - 串行成功写入：数据区域“定义且一致”，所有客户端读取结果相同；
 - 并发成功写入：数据区域“一致但未定义”，客户端读取结果相同但可能是多写入者数据片段的混合；
 - 写入失败：数据区域“不一致”，不同客户端可能读取到不同结果。
- **应用适配方案**：通过追加写入而非覆盖、定期 checkpoint、记录自检校验和等方式，确保应用能识别有效数据区域。

3. 原子追加与快照

- **原子记录追加 (Record Append)**：支持多客户端并发向同一文件追加数据，GFS 保证每个客户端的记录至少原子写入一次，无需客户端额外同步。当数据块剩余空间不足时，会自动填充至 64MB 并引导客户端写入新数据块。
- **快照 (Snapshot)**：基于写时复制 (Copy-on-Write) 实现，创建文件或目录树副本时仅复制元数据，不拷贝实际数据。当客户端首次修改快照关联的数据块时，才会创建新数据块副本，实现低成本快速快照。

4. 容错与数据完整性保障

GFS 构建了多层次容错机制，应对硬件故障和数据损坏：

- **副本容错**：数据块副本分布在不同机架，即使整个机架离线，仍能通过其他机架的副本恢复数据；Master 会持续监控副本数量，当副本数低于阈值时自动触发重复制 (Re-replication)。
- **Master 高可用**：通过操作日志和 checkpoint 副本实现故障恢复，影子 Master (Shadow Master) 提供只读访问能力，即使主 Master 离线，仍可支持读操作。
- **数据完整性校验**：每个 Chunkserver 为 64KB 数据块维护 32 位校验和，读取时校验数据完整性，发现损坏则通过其他副本恢复，并删除损坏副本。
- **垃圾回收**：文件删除后不会立即释放空间，而是重命名为含时间戳的隐藏文件，Master 定期扫描并清理超过保留期（默认 3 天）的文件及孤儿数据块，兼顾可靠性与存储空间复用。

5. 读写流程优化

- **读取流程**：Client 先通过文件名和偏移量计算数据块索引，向 Master 请求该数据块的副本位置；Client 缓存位置信息，直接向最近的 Chunkserver 读取数据，后续同一数据块读取无需再次请求 Master。
- **写入流程**：采用“控制流与数据流分离”策略：
 - a. Client 向 Master 获取数据块的主副本（Primary）和从副本（Secondary）位置；
 - b. Client 将数据推送到所有副本的 LRU 缓存（可按网络拓扑优化推送顺序）；
 - c. 所有副本确认接收数据后，Client 向 Primary 发送写入请求；
 - d. Primary 按序列号排序写入操作，同步至所有 Secondary，待所有副本确认后返回成功。



一、核心问题：分布式共识的要求

Paxos要解决的核心是**分布式共识问题**：让一组进程对某个提议的值达成一致选择，同时满足**安全要求**（无论系统发生何种非拜占庭故障，都必须遵守），并尽可能满足**活性要求**（系统

正常时，最终能达成共识）。

1. 安全三原则

- 只有被提议的值才能被选中；
- 最终只能有一个值被选中；
- 进程不会得知一个未被真正选中的值。

2. 系统模型假设

- 异步通信：消息传递时长任意、可丢失、可重复，但不会被篡改；
- 非拜占庭故障：节点只会崩溃停止，不会恶意篡改数据；
- 节点可重启：但需要持久化存储关键信息，重启后能恢复状态。

3. 三个核心角色

算法将参与方划分为三个独立角色（实际实现中，一个进程可兼任多个角色）：

- **提议者（Proposer）**：提出待共识的值；
- **接受者（Acceptor）**：决定是否接受提议，是共识的核心决策方；
- **学习者（Learner）**：获取被选中的值，同步共识结果。

二、核心逻辑：如何选择一个共识值

这是Paxos的核心部分，论文通过逐步推导规则，最终得出**两阶段提案算法**，核心是通过**多数派（Majority）**机制和**提案编号**保证一致性，同时解决了单接受者的单点故障问题。

1. 从单接受者到多接受者的演进

- 单接受者：简单但存在单点故障，接受者崩溃后系统无法继续；
- 多接受者：引入**多数派规则**（超过半数接受者同意则值被选中），因任意两个多数派必有交集，能保证唯一值被选中。

2. 核心规则的逐步推导

为解决多接受者下的一致性问题的，论文推导出一系列递进的规则，最终形成可落地的约束：

1. **P1**：接受者必须接受它收到的第一个提案；

问题：多个提议者同时提案时，可能出现每个接受者各接受不同值，无任何值获多数派支持的情况。

2. **P2**：若值U被选中，所有更高编号的提案若被选中，其值也必须是U；

核心：通过提案编号的全序性，保证一旦有值达成共识，后续提案不会改变该值。

3. **P2a/P2b**：对P2的强化，最终推导出**P2c**（核心不变量）：

若提议者发出编号为 n 、值为 v 的提案，则存在一个多数派接受者集合 S ，要么 S 中无接受者接受过编号小于 n 的提案，要么 v 是 S 中接受者接受的**编号小于 n 的最高编号提案的值**。

P2c是整个算法的核心约束，保证了提案的一致性。

3. 最终的两阶段提案算法

为满足P2c，提议者和接受者通过**准备阶段（Phase 1）**和**接受阶段（Phase 2）**完成提案，且接受者仅需持久化两个关键信息：**已接受的最高编号提案、已响应的最高编号准备请求**。

阶段1：准备（Prepare）

1. 提议者选择一个**唯一的提案编号 n** ，向**多数派接受者**发送准备请求，要求：
 - 接受者承诺不再接受编号小于 n 的提案；
 - 接受者返回其已接受的编号小于 n 的**最高编号提案**（若有）。
2. 接受者若收到的 n 大于其已响应的最高准备请求编号，则回复请求并记录 n ，否则忽略。

阶段2：接受（Accept）

1. 提议者若收到**多数派接受者的回复**，则确定提案值 v ：
 - 若回复中有已接受的提案， v 为其中**最高编号提案的值**；
 - 若无已接受的提案， v 可为提议者任意选择的值。

随后提议者向接受者发送**接受请求**（含编号 n 和值 v ）。

2. 接受者若未向更高编号的准备请求做出承诺，则接受该提案，否则忽略。

关键优化：接受者可忽略已承诺过更高编号的准备请求，无需重复响应，减少无效通信。

三、共识结果的学习：Learner如何获取选中的值

学习者的核心是确认**某个提案被多数派接受者接受**，论文提出三种实现方式，兼顾效率和可靠性：

1. **直接通知：**接受者接受提案后，直接向所有学习者发送通知；

优点：实时性高；缺点：通信量大（接受者数 $\times$ 学习者数）。

2. **指定学习者：**接受者仅向一个**特殊学习者**发送通知，由其统一转发给其他学习者；

优点：通信量小（接受者数+学习者数）；缺点：特殊学习者存在单点故障。

3. **多指定学习者：**接受者向一组特殊学习者发送通知，再由其转发；

优点：兼顾可靠性和通信效率；缺点：通信复杂度略高。

异常处理：若因消息丢失导致学习者未获取共识结果，可让提议者重新发起提案，通过新提案的执行同步共识结果。

四、活性保证：如何避免算法死锁

Paxos的安全要求在任何情况下都能满足，但可能出现**活锁**：两个提议者交替发起更高编号的提案，彼此的接受请求被对方的准备请求忽略，导致无提案被选中。

解决方法：选举唯一的主提议者（Distinguished Proposer）

- 让**单个提议者**成为唯一的提案发起方，若其能与多数派接受者正常通信，必然能成功发起提案；
- 主提议者若发现有更高编号的提案，可放弃当前提案并重新发起更高编号的提案，最终能达成共识。

关键结论

- 主提议者的选举可通过**超时机制**或**随机算法**实现（参考Fischer-Lynch-Patterson不可能结果：异步系统中，无随机/时间的选举算法无法保证可靠性）；
- 即使选举失败（无主/多个主），**安全要求仍能满足**，只是可能暂时无法达成共识，选举仅为保证**活性**。



实验记录



实验要求

- ☐ rpc.go 创建需要用到结构体
- ☐ coordinator.go 协调worker线程，为worker线程分配任务
- ☐ worker.go 调用map函数、reduce函数处理数据



实现细节

1. rpc包接收到新请求时，会创建新协程来调用注册的handler处理请求
2. rpc包在传输数据的时候，请求时只传送Args结构体数据，响应时只返回Reply结构体数据
3. map函数处理完的数据应该按照mapID 和 reduce分区存储在中间文件中，避免同一reduce分区的不同mapID的结果存放在同一中间文件中，这样会相互影响，不能保证幂等性
4. 中间文件在创建时应该先创建临时文件，后面再通过原子操作重命名文件，而不是append写入同一中间文件中。避免任务处理超时，coordinator将该任务重新分配给其他worker线程，多线程操作中间文件不能保证处理结果的幂等性
5. coordinator线程在退出前应该保证所有worker线程都已退出，为了同步线程状态，我们引入了心跳机制，这样可以保证线程状态的同步，并清理崩溃的worker线程（15s没接收到心跳则认为worker线程崩溃）
6. coordinator线程正常退出前，应该将map中间文件删除掉
7. 心跳机制：worker线程通过rpc进行心跳请求，coordinator线程接收后更新对应workerID的最新心跳时间。如果超过15s没有心跳则判定为worker线程崩溃

Worker线程：

- a. worker线程中使用雪花算法生成UUID作为workerID，创建一个chan，开启一个协程，通过rpc心跳请求注册到coordinator线程中，并开启协程每3s发起一次心跳，当心跳不可达 或 coordinator线程返回stop信号时 关闭chan 退出协程
- b. 当rpc请求任务的请求不可达时，认为coordinator线程崩溃，关闭chan并退出

Coordinator线程：

- a. coordinator线程开启协程检查worker线程的心跳有没有超时，如果超时了就将该workerID移除哈希表中，如果coordinator中所有任务完成 且 哈希表为空 就清空中间文件并退出该协程

- b. 当worker线程的心跳请求发送了coordinator中的处理函数中时，我们检查所有任务是否完成，已完成则返回退出信号给worker线程，未完成则看该线程是否为新线程，是否注册在coordinator的哈希表中，未注册就注册，紧接着更新心跳时间

```
rpc.go

1  package mr
2
3  import (
4      "os"
5      "strconv"
6      "sync"
7      "time"
8  )
9
10 // ----- 雪花算法生成 WorkerID -----
11
12 const (
13     sequenceBits uint8 = 12
14     workerBits   uint8 = 10
15     maxSequence  int64 = -1 ^ (-1 << sequenceBits)
16     timeShift    uint8 = workerBits + sequenceBits
17     workerShift   = sequenceBits
18     epoch        int64 = 1700000000000 // 自定义起始时间戳 ms
19 )
20
21 type snowflake struct {
22     mu      sync.Mutex
23     lastStamp int64
24     workerID int64
25     sequence int64
26 }
27
28 var sf = &snowflake{workerID: int64(os.Getpid()) & 0x3FF}
29
30 func GenerateID() int64 {
31     sf.mu.Lock()
32     defer sf.mu.Unlock()
33     now := time.Now().UnixMilli()
34     if now == sf.lastStamp {
35         sf.sequence = (sf.sequence + 1) & maxSequence
36         if sf.sequence == 0 {
37             // 序列号溢出，等到下一毫秒
38             for now <= sf.lastStamp {
39                 now = time.Now().UnixMilli()
40             }
41         }
42     }
43     return sf.workerID << timeShift | sf.sequence << workerShift | now << timeShift
```

```

41     }
42     } else {
43         sf.sequence = 0
44     }
45     sf.lastStamp = now
46     return (now-epoch)<<timeShift | sf.workerID<<workerShift | sf.sequence
47 }
48
49 // ----- RPC 数据结构 -----
50
51 type Args struct {
52     Addr string // worker 汇报完成的任务地址
53 }
54
55 type Reply struct {
56     T      string // map / reduce / wait / exit
57     N      int     // reduce 任务数量
58     Addr   string // 任务标识
59     Index  int     // reduce 任务索引
60     MapIndex int    // map 任务索引
61 }
62
63 type HeartbeatArgs struct {
64     WorkerID int64
65 }
66
67 type HeartbeatReply struct {
68     Stop bool // true 表示所有任务已完成, worker 应停止心跳并退出
69 }
70
71 func coordinatorSock() string {
72     s := "/var/tmp/824-mr-"
73     s += strconv.Itoa(os.Getuid())
74     return s
75 }

```

coordinator

```

1 package mr
2
3 import (
4     "fmt"
5     "log"
6     "path/filepath"
7     "sync"
8     "time"

```



```

9  )
10 import "net"
11 import "os"
12 import "net/rpc"
13 import "net/http"
14
15 var mu sync.Mutex
16
17 type workerInfo struct {
18     lastSeen time.Time
19 }
20
21 type Coordinator struct {
22     // 输入文件列表 (map 任务)
23     mf []string
24     // 任务状态 0 待处理 1 执行中 2 已完成
25     book map[string]int
26     // reduce 任务数量
27     r int
28     // map 完成数量
29     mtnum int
30     // reduce 完成数量
31     rtnum int
32     // 活跃 worker 列表 workerID -> 最后心跳时间
33     workers map[int64]*workerInfo
34     // 是否已完成所有任务
35     done bool
36 }
37
38 // ----- 任务分配 -----
39
40 func (c *Coordinator) CoordinatorHandler(args *Args, reply *Reply) error {
41     mu.Lock()
42     defer mu.Unlock()
43     if c.mtnum < len(c.mf) {
44         return c.maptask(args, reply)
45     }
46     return c.reducesetask(args, reply)
47 }
48
49 func (c *Coordinator) maptask(args *Args, reply *Reply) error {
50     // 汇报已完成的任务
51     if v, ok := c.book[args.Addr]; ok && v == 1 {
52         c.book[args.Addr] = 2
53         c.mtnum++
54     }
55     if c.mtnum == len(c.mf) {

```

```

56     reply.T = "wait"
57     return nil
58 }
59 for i, addr := range c.mf {
60     if c.book[addr] == 0 {
61         c.book[addr] = 1
62         reply.T = "map"
63         reply.N = c.r
64         reply.Addr = addr
65         reply.MapIndex = i
66         go c.timeadd(addr)
67         return nil
68     }
69 }
70 reply.T = "wait"
71 return nil
72 }
73
74 func (c *Coordinator) reducetask(args *Args, reply *Reply) error {
75     // 汇报已完成的任务
76     if v, ok := c.book[args.Addr]; ok && v == 1 {
77         c.book[args.Addr] = 2
78         c.rtnum++
79     }
80     if c.rtnum == c.r {
81         reply.T = "exit"
82         c.done = true
83         return nil
84     }
85     for i := 0; i < c.r; i++ {
86         key := fmt.Sprintf("reduce-%d", i)
87         if c.book[key] == 0 {
88             c.book[key] = 1
89             reply.T = "reduce"
90             reply.Index = i
91             reply.Addr = key
92             go c.timeadd(key)
93             return nil
94         }
95     }
96     reply.T = "wait"
97     return nil
98 }
99
100 // ----- 超时重置 -----
101
102 func (c *Coordinator) timeadd(key string) {

```

```
103     time.Sleep(10 * time.Second)
104     mu.Lock()
105     defer mu.Unlock()
106     if c.book[key] == 1 {
107         c.book[key] = 0
108         fmt.Printf("任务超时, 重新分配: %v\n", key)
109     }
110 }
111
112 // ----- 心跳 -----
113
114 func (c *Coordinator) Heartbeat(args *HeartbeatArgs, reply *HeartbeatReply)
115 error {
116     mu.Lock()
117     defer mu.Unlock()
118     if c.done {
119         // 所有任务完成, 通知 worker 停止
120         reply.Stop = true
121         delete(c.workers, args.WorkerID)
122         return nil
123     }
124     // 更新最后心跳时间
125     if _, ok := c.workers[args.WorkerID]; !ok {
126         c.workers[args.WorkerID] = &workerInfo{}
127     }
128     c.workers[args.WorkerID].lastSeen = time.Now()
129     reply.Stop = false
130     return nil
131 }
132
133 // 后台协程: 定期清理失联 worker (超过 15 秒无心跳视为已死亡)
134 func (c *Coordinator) watchWorkers() {
135     for {
136         time.Sleep(5 * time.Second)
137         mu.Lock()
138         for id, info := range c.workers {
139             if time.Since(info.lastSeen) > 15*time.Second {
140                 fmt.Printf("worker %d 失联, 移除\n", id)
141                 delete(c.workers, id)
142             }
143         }
144         // 所有任务完成且没有活跃 worker, 清理中间文件
145         if c.done && len(c.workers) == 0 {
146             mu.Unlock()
147             c.cleanup()
148             return
149         }
150     }
151 }
```

```
149     mu.Unlock()
150 }
151 }
152
153 // 清理所有中间文件
154 func (c *Coordinator) cleanup() {
155     files, _ := filepath.Glob("map-result-*")
156     for _, f := range files {
157         os.Remove(f)
158         fmt.Printf("清理中间文件: %v\n", f)
159     }
160     fmt.Println("所有中间文件已清理, coordinator 退出")
161 }
162
163 // ----- Done -----
164
165 func (c *Coordinator) Done() bool {
166     mu.Lock()
167     defer mu.Unlock()
168     // 所有任务完成 且 没有活跃 worker
169     return c.done && len(c.workers) == 0
170 }
171
172 // ----- 启动 -----
173
174 func (c *Coordinator) server() {
175     rpc.Register(c)
176     rpc.HandleHTTP()
177     sockname := coordinatorSock()
178     os.Remove(sockname)
179     l, e := net.Listen("unix", sockname)
180     if e != nil {
181         log.Fatal("listen error:", e)
182     }
183     go http.Serve(l, nil)
184 }
185
186 func MakeCoordinator(files []string, nReduce int) *Coordinator {
187     c := Coordinator{}
188     c.mf = append(c.mf, files...)
189     c.r = nReduce
190     c.book = make(map[string]int)
191     c.workers = make(map[int64]*workerInfo)
192
193     for _, v := range c.mf {
194         c.book[v] = 0
195     }
```

```

196     for i := 0; i < nReduce; i++ {
197         c.book[fmt.Sprintf("reduce-%d", i)] = 0
198     }
199
200     go c.watchWorkers()
201     c.server()
202     return &c
203 }

```

worker.go

```

1  package mr
2
3  import (
4      "bufio"
5      "fmt"
6      "io/ioutil"
7      "os"
8      "path/filepath"
9      "sort"
10     "strconv"
11     "strings"
12     "time"
13 )
14 import "log"
15 import "net/rpc"
16 import "hash/fnv"
17
18 type KeyValue struct {
19     Key    string
20     Value  string
21 }
22
23 func ihash(key string) int {
24     h := fnv.New32a()
25     h.Write([]byte(key))
26     return int(h.Sum32() & 0x7fffffff)
27 }
28
29 func Worker(mapf func(string, string) []KeyValue,
30     reducef func(string, []string) string) {
31
32     workerID := GenerateID()
33
34     // 后台心跳协程
35     stopHeartbeat := make(chan struct{})

```



```

36     go func() {
37         for {
38             select {
39                 case <-stopHeartbeat:
40                     return
41                 case <-time.After(3 * time.Second):
42                     hArgs := &HeartbeatArgs{WorkerID: workerID}
43                     hReply := &HeartbeatReply{}
44                     ok := call("Coordinator.Heartbeat", hArgs, hReply)
45                     if !ok || hReply.Stop {
46                         // coordinator 已完成或不可达, 通知主循环退出
47                         close(stopHeartbeat)
48                         return
49                     }
50             }
51         }
52     }()
53
54     args, reply := &Args{}, &Reply{}
55
56     for {
57         // 检查心跳协程是否已通知退出
58         select {
59             case <-stopHeartbeat:
60                 fmt.Println("收到退出信号, worker 退出")
61                 return
62             default:
63             }
64
65             fmt.Printf("reply : %+v\n", reply)
66             switch reply.T {
67             case "map":
68                 args.Addr = reply.Addr
69                 if err := maptask(mapf, reply); err != nil {
70                     log.Printf("maptask 处理失败 %v: %v", reply.Addr, err)
71                     args.Addr = "" // 不汇报完成, 让超时机制重新分配
72                 }
73                 if !Call(args, reply) {
74                     close(stopHeartbeat)
75                     return
76                 }
77
78             case "reduce":
79                 args.Addr = reply.Addr
80                 if err := reducetask(reducef, reply); err != nil {
81                     log.Printf("reducetask 处理失败 %v: %v", reply.Addr, err)
82                     args.Addr = ""

```

```
83         }
84         if !Call(args, reply) {
85             close(stopHeartbeat)
86             return
87         }
88
89         case "wait":
90             time.Sleep(3 * time.Second)
91             args.Addr = ""
92             if !Call(args, reply) {
93                 close(stopHeartbeat)
94                 return
95             }
96
97         case "exit":
98             fmt.Println("所有任务完成, worker 正常退出")
99             close(stopHeartbeat)
100             return
101
102         default:
103             args.Addr = ""
104             if !Call(args, reply) {
105                 close(stopHeartbeat)
106                 return
107             }
108     }
109 }
110 }
111
112 // ----- map 任务 -----
113
114 func maptask(mapf func(string, string) []KeyValue, reply *Reply) error {
115     file, err := os.Open(reply.Addr)
116     if err != nil {
117         return fmt.Errorf("cannot open %v: %w", reply.Addr, err)
118     }
119     content, err := ioutil.ReadAll(file)
120     file.Close()
121     if err != nil {
122         return fmt.Errorf("cannot read %v: %w", reply.Addr, err)
123     }
124
125     kva := mapf(reply.Addr, string(content))
126
127     // 先写临时文件
128     tmpFiles := make([]*os.File, reply.N)
129     tmpNames := make([]string, reply.N)
```

```

130     for i := 0; i < reply.N; i++ {
131         tmp, err := os.CreateTemp("", fmt.Sprintf("map-tmp-%d-%d-*",
reply.MapIndex, i))
132         if err != nil {
133             return err
134         }
135         tmpFiles[i] = tmp
136         tmpNames[i] = tmp.Name()
137     }
138     defer func() {
139         for _, f := range tmpFiles {
140             if f != nil {
141                 f.Close()
142             }
143         }
144     }()
145
146     for _, kv := range kva {
147         index := ihash(kv.Key) % reply.N
148         if _, err = fmt.Fprintf(tmpFiles[index], "%v\t%v\n", kv.Key, kv.Value);
err != nil {
149             return err
150         }
151     }
152
153     // 全部写完后原子 rename, 文件名带 mapIndex 保证不同 map 任务互不覆盖
154     for i := 0; i < reply.N; i++ {
155         tmpFiles[i].Close()
156         tmpFiles[i] = nil
157         dest := fmt.Sprintf("map-result-%d-%d", reply.MapIndex, i)
158         if err := os.Rename(tmpNames[i], dest); err != nil {
159             return err
160         }
161     }
162     return nil
163 }
164
165 // ----- reduce 任务 -----
166
167 func reducetask(reducef func(string, []string) string, reply *Reply) error {
168     // 读取所有属于该 reduce 任务的中间文件
169     pattern := fmt.Sprintf("map-result-*-%d", reply.Index)
170     files, err := filepath.Glob(pattern)
171     if err != nil || len(files) == 0 {
172         return fmt.Errorf("no intermediate files for reduce-%d", reply.Index)
173     }
174

```

```
175     var kva []KeyValue
176     for _, fname := range files {
177         f, err := os.Open(fname)
178         if err != nil {
179             return err
180         }
181         scanner := bufio.NewScanner(f)
182         scanner.Buffer(make([]byte, 1024*1024), 1024*1024)
183         for scanner.Scan() {
184             line := scanner.Text()
185             parts := strings.SplitN(line, "\t", 2)
186             if len(parts) != 2 {
187                 continue
188             }
189             kva = append(kva, KeyValue{parts[0], parts[1]})
190         }
191         f.Close()
192         if err := scanner.Err(); err != nil {
193             return fmt.Errorf("scanner error %v: %w", fname, err)
194         }
195     }
196
197     sort.Slice(kva, func(i, j int) bool {
198         return kva[i].Key < kva[j].Key
199     })
200
201     // 先写临时文件, 完成后原子 rename
202     oname := "mr-out-" + strconv.Itoa(reply.Index)
203     ofile, err := os.CreateTemp("", oname+"-tmp-*")
204     if err != nil {
205         return err
206     }
207     defer func() {
208         if ofile != nil {
209             ofile.Close()
210         }
211     }()
212
213     i := 0
214     for i < len(kva) {
215         j := i + 1
216         for j < len(kva) && kva[j].Key == kva[i].Key {
217             j++
218         }
219         values := make([]string, 0, j-i)
220         for k := i; k < j; k++ {
221             values = append(values, kva[k].Value)
```

```

222     }
223     output := reducef(kva[i].Key, values)
224     if _, err = fmt.Fprintf(ofile, "%v %v\n", kva[i].Key, output); err !=
nil {
225         return err
226     }
227     i = j
228 }
229
230 tmpName := ofile.Name()
231 ofile.Close()
232 ofile = nil
233 if err := os.Rename(tmpName, oname); err != nil {
234     return err
235 }
236 return nil
237 }
238
239 // ----- RPC -----
240
241 func Call(args *Args, reply *Reply) bool {
242     ok := call("Coordinator.CoordinatorHandler", args, reply)
243     if ok {
244         fmt.Printf("RPC 响应成功: %+v\n", reply)
245     } else {
246         fmt.Println("coordinator 不可达, worker 退出")
247     }
248     return ok
249 }
250
251 func call(rpcname string, args interface{}, reply interface{}) bool {
252     sockname := coordinatorSock()
253     c, err := rpc.DialHTTP("unix", sockname)
254     if err != nil {
255         log.Printf("dialing failed: %v", err)
256         return false
257     }
258     defer c.Close()
259     err = c.Call(rpcname, args, reply)
260     if err == nil {
261         return true
262     }
263     fmt.Println(err)
264     return false
265 }

```